MATH

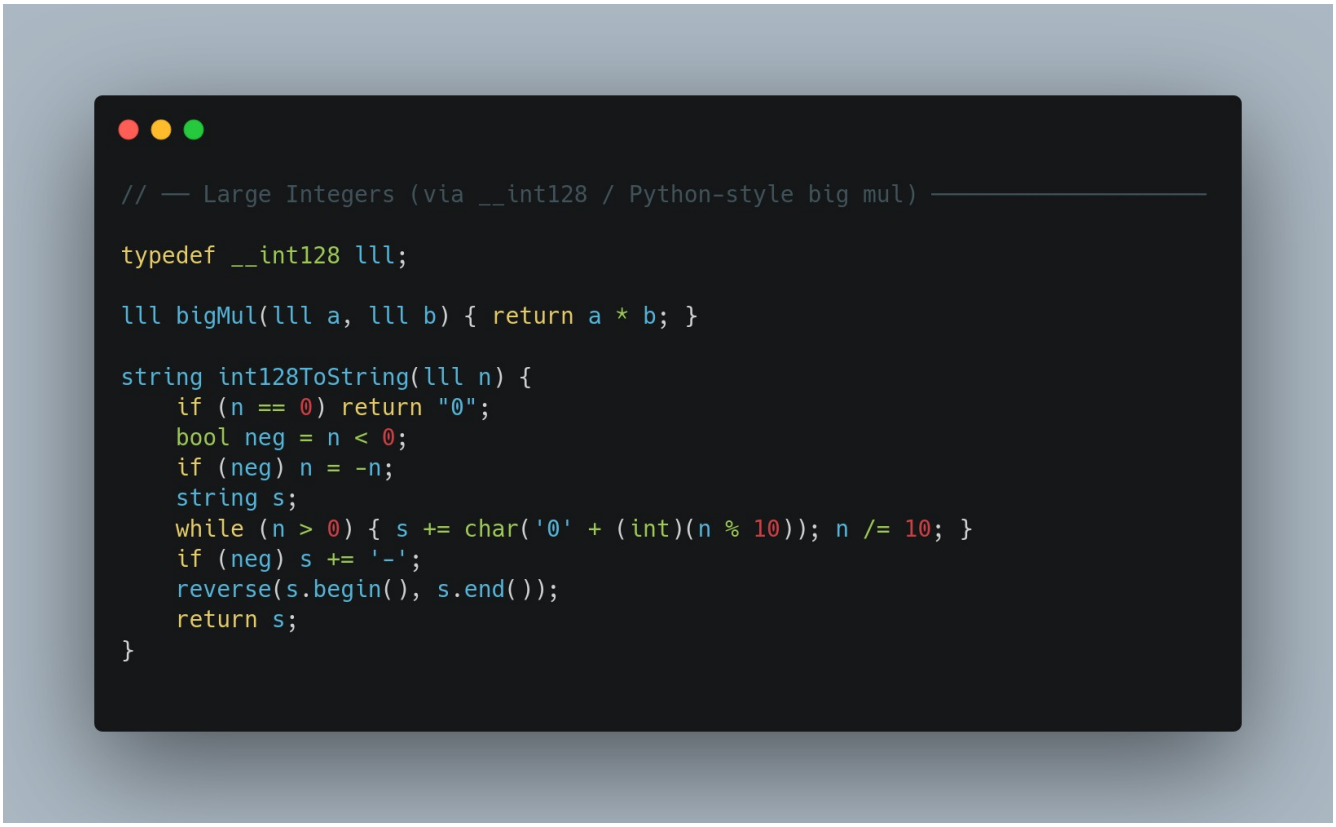
```cpp
// ── Primes ──────────────────────────────────────────────

bool isPrime(long long n) {
    if (n < 2) return false;
    if (n < 4) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (long long i = 5; i * i <= n; i += 6)
        if (n % i == 0 || n % (i + 2) == 0) return false;
    return true;
}

vector<int> sieve(int limit) {
    vector<bool> is_prime(limit + 1, true);
    vector<int> primes;
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; i <= limit; i++) {
        if (is_prime[i]) {
            primes.push_back(i);
            for (long long j = (long long)i * i; j <= limit; j += i)
                is_prime[j] = false;
        }
    }
    return primes;
}

vector<long long> primeFactors(long long n) {
    vector<long long> factors;
    for (long long i = 2; i * i <= n; i++)
        while (n % i == 0) { factors.push_back(i); n /= i; }
    if (n > 1) factors.push_back(n);
    return factors;
}
```

MATH



```cpp
// ── Large Integers (via __int128 / Python-style big mul) ──────────────────

typedef __int128 lll;

lll bigMul(lll a, lll b) { return a * b; }

string int128ToString(lll n) {
    if (n == 0) return "0";
    bool neg = n < 0;
    if (neg) n = -n;
    string s;
    while (n > 0) { s += char('0' + (int)(n % 10)); n /= 10; }
    if (neg) s += '-';
    reverse(s.begin(), s.end());
    return s;
}
```

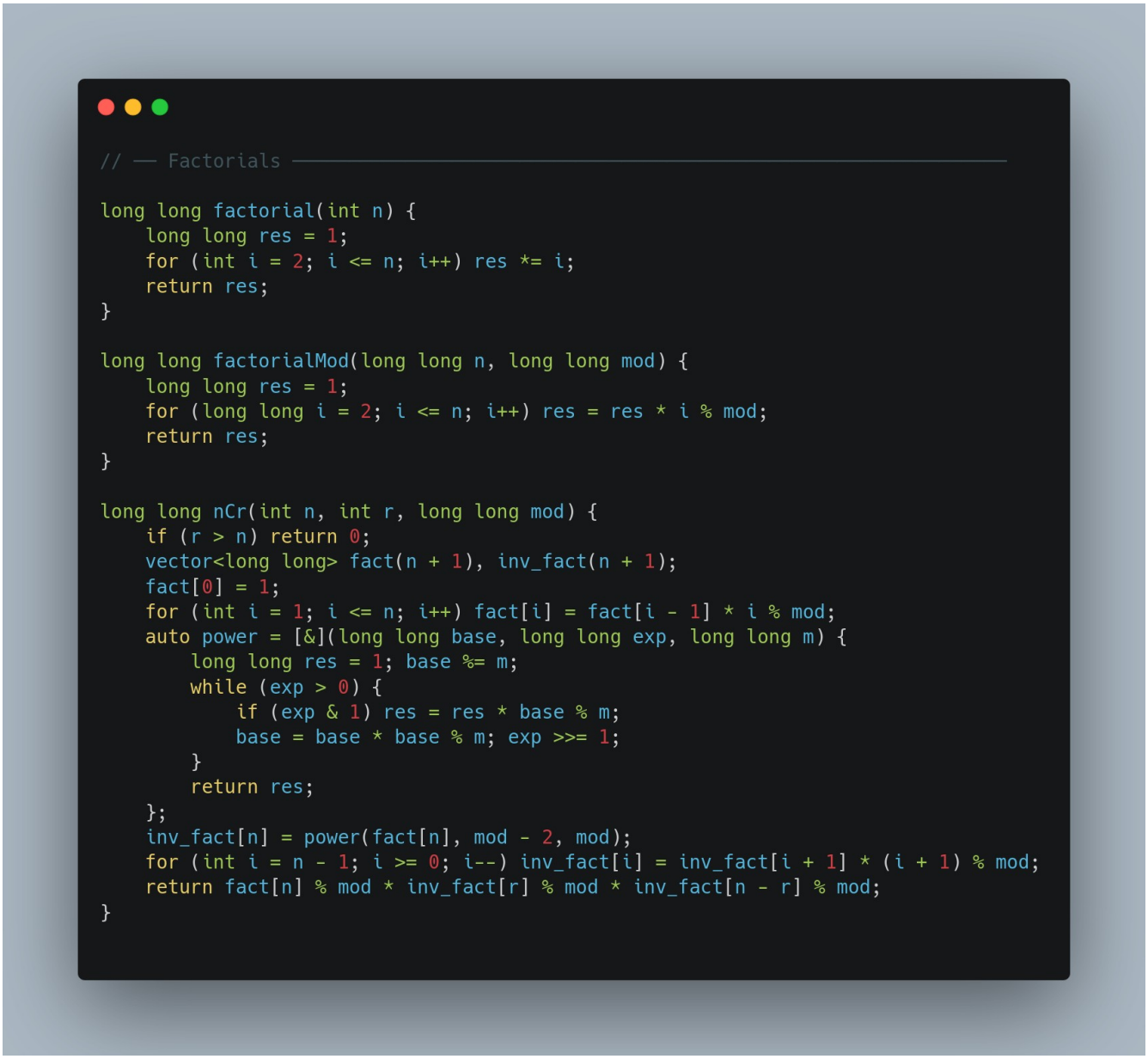
MATH

```cpp
// ── Permutations ──────────────────────────────────────────────

vector<vector<int>> permutations(vector<int> arr) {
    vector<vector<int>> result;
    sort(arr.begin(), arr.end());
    do result.push_back(arr);
    while (next_permutation(arr.begin(), arr.end()));
    return result;
}

long long countPermutations(int n, int r) {
    long long res = 1;
    for (int i = 0; i < r; i++) res *= (n - i);
    return res;
}
```

MATH



```cpp
// ── Factorials ──────────────────────────────────────────────

long long factorial(int n) {
    long long res = 1;
    for (int i = 2; i <= n; i++) res *= i;
    return res;
}

long long factorialMod(long long n, long long mod) {
    long long res = 1;
    for (long long i = 2; i <= n; i++) res = res * i % mod;
    return res;
}

long long nCr(int n, int r, long long mod) {
    if (r > n) return 0;
    vector<long long> fact(n + 1), inv_fact(n + 1);
    fact[0] = 1;
    for (int i = 1; i <= n; i++) fact[i] = fact[i - 1] * i % mod;
    auto power = [&](long long base, long long exp, long long m) {
        long long res = 1; base %= m;
        while (exp > 0) {
            if (exp & 1) res = res * base % m;
            base = base * base % m; exp >>= 1;
        }
        return res;
    };
    inv_fact[n] = power(fact[n], mod - 2, mod);
    for (int i = n - 1; i >= 0; i--) inv_fact[i] = inv_fact[i + 1] * (i + 1) % mod;
    return fact[n] % mod * inv_fact[r] % mod * inv_fact[n - r] % mod;
}
```

MATH

```cpp
// ── Fibonacci ──────────────────────────────────────────────────────

long long fibonacci(int n) {
    if (n <= 1) return n;
    long long a = 0, b = 1;
    for (int i = 2; i <= n; i++) { long long c = a + b; a = b; b = c; }
    return b;
}

long long fibonacciMod(long long n, long long mod) {
    if (n <= 1) return n;
    auto matMul = [&](vector<vector<long long>> A, vector<vector<long long>> B) {
        int sz = A.size();
        vector<vector<long long>> C(sz, vector<long long>(sz, 0));
        for (int i = 0; i < sz; i++)
            for (int k = 0; k < sz; k++)
                for (int j = 0; j < sz; j++)
                    C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % mod;
        return C;
    };
    auto matPow = [&](auto& self, vector<vector<long long>> M, long long p) -> vector<vector<long
long>> {
        int sz = M.size();
        vector<vector<long long>> R(sz, vector<long long>(sz, 0));
        for (int i = 0; i < sz; i++) R[i][i] = 1;
        while (p > 0) {
            if (p & 1) R = matMul(R, M);
            M = matMul(M, M); p >>= 1;
        }
        return R;
    };
    vector<vector<long long>> M = {{1, 1}, {1, 0}};
    auto R = matPow(matPow, M, n);
    return R[0][1];
}
```

MATH

```cpp
// ── Modulo ──────────────────────────────────────────────

long long modPow(long long base, long long exp, long long mod) {
    long long result = 1;
    base %= mod;
    while (exp > 0) {
        if (exp & 1) result = result * base % mod;
        base = base * base % mod;
        exp >>= 1;
    }
    return result;
}

long long modInverse(long long a, long long mod) {
    return modPow(a, mod - 2, mod);
}

long long extGCD(long long a, long long b, long long& x, long long& y) {
    if (b == 0) { x = 1; y = 0; return a; }
    long long x1, y1;
    long long g = extGCD(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}
```